

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a vast, blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene and expansive feel.

DL for ENG

## LECTURE 4.1

# From Perceptron to Neural Network

---

**Remus Teodorescu**

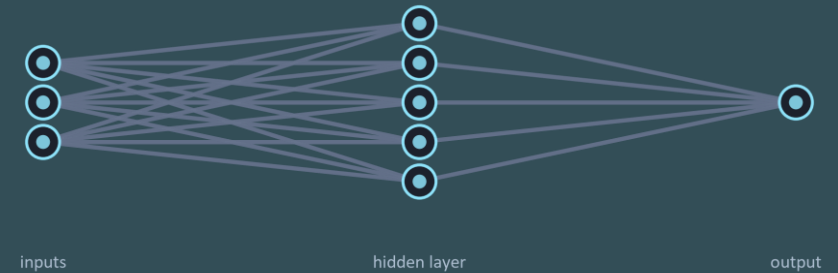
*With support from Research Assistant Noman Khan - nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

*linear regression · perceptron · activation · universal approximation*

### ONE HIDDEN LAYER



*An input layer, one hidden layer of neurons, an output layer. This lecture defines each of them, states what the whole can represent, and gives a first rule for how many neurons a problem needs.*

# Learning Objectives

## SECTIONS, IN ORDER

- 1 **Supervised learning** — the four objects: data, model, parameters, loss
- 2 **Linear regression** — the simplest model, and its loss surface
- 3 **Perceptron and neuron** — the 1958 unit and the modern one, side by side
- 4 **Neuron limitations** — one neuron draws one line; XOR needs a layer
- 5 **Activation function** — which nonlinearity, and what its derivatives carry
- 6 **Neural networks** — one hidden layer; linear regions; the theorem
- 7 **Inputs and outputs** — several of each, from one shared hidden layer
- 8 **Sizing a neural network** — width and depth, chosen in practice
- 9 **Challenges** — what goes wrong first, and what to check

## WHAT WE SKIP, AND WHERE IT IS TAUGHT

|                       |                                     |
|-----------------------|-------------------------------------|
| <b>training</b>       | how the parameters are found — L6.1 |
| <b>generalisation</b> | unseen data and capacity — L4.2     |
| <b>depth</b>          | more than one hidden layer — L4.2   |
| <b>the loss</b>       | why squared error — L6.1            |

*Every curve in this lecture is already fitted. Of the three ingredients from lecture three, the loss and the optimiser are held fixed today and only the function is studied.*

# Recap

## THE TEN QUESTIONS OF L3.2 — AI APPLICATION EXAMPLES

- 1** What are the four questions to ask of any reported deep-learning result, and what does each one test? *L3.2; UDL ch. 1.4*
- 2** What are the three classic vision tasks, and why does the third need more data than the first? *UDL ch. 10.5*
- 3** What does a large language model compute at each step, and what does that make it good and bad at? *UDL ch. 12.5 – 12.8*
- 4** What problem did AlphaFold solve, what data did it learn from, and what does it still not give you? *Jumper et al. (2021), Nature 596*
- 5** What is GraphCast, why is a graph the right structure for it, and what does it depend on at every run? *Lam et al. (2023), Science 382*
- 6** What did the fusion-control result show, and which of the four questions does it answer least well? *Degrave et al. (2022), Nature 602; UDL ch. 19*
- 7** What is a surrogate model, when is training one worth the cost, and what is the cost? *L3.2; UDL ch. 1.4*
- 8** Where does the data for predictive maintenance come from, and what decides whether a model is deployable? *L3.2 slides on engineering; UDL ch. 1.4*
- 9** What separates a shipped system from an assisting one and from a demonstration, and how would you tell from the published record? *L3.2, the honest ledger; UDL ch. 1.4*
- 10** Which ethical constraint changes an engineering decision in your own field, and how would you state it as a requirement? *UDL ch. 21*

# Supervised Learning

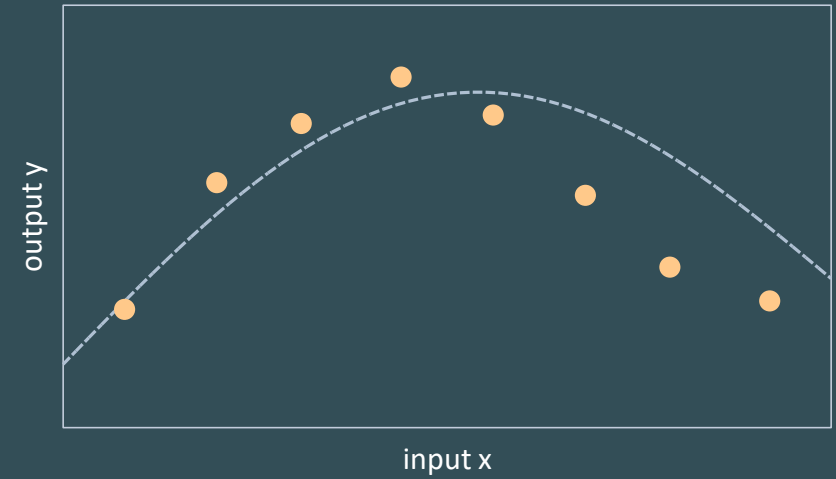
## FOUR OBJECTS

- **N input-output pairs** — the data
- **a model  $f$**  — takes an input and a parameter vector, returns a prediction
- **a loss** — scores how wrong the predictions are, in one number
- **every engineering choice** — is a choice of  $f$  or of the loss
- **training once, inference many times** — the case for a surrogate model

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_{k=1}^N \ell(f(x_k, \theta), y_k)$$

$\hat{\theta}$  the fitted parameters,  $f$  the model,  $\ell$  the loss,  $(x_k, y_k)$  the  $k$ th of  $N$  pairs

## DATA AND THE FUNCTION BEHIND IT



*Amber dots are the measured pairs. The dashed line is the function that produced them. The model  $f$  has to be flexible enough to follow it.*

# Linear Regression

## MODEL AND LOSS

- **one input, one output** — two parameters, intercept and slope
- **least squares** — minimise the summed squared vertical distances
- **the baseline** — every model in this lecture is measured against it

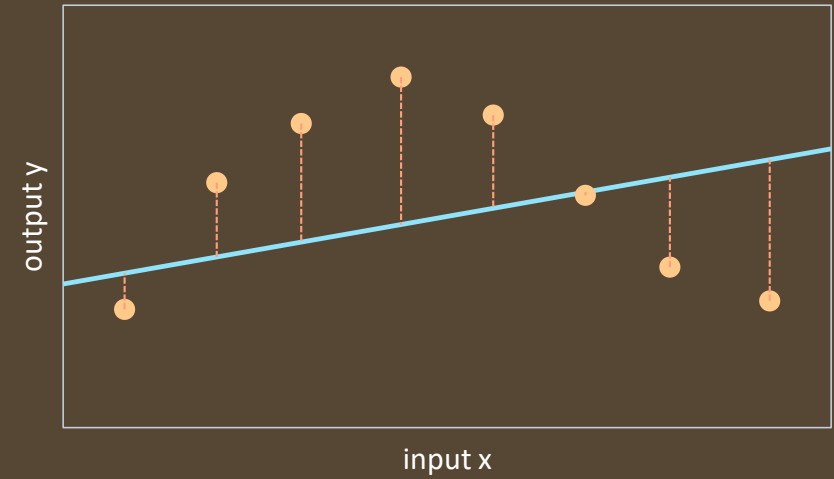
$$f(x, \theta) = \theta_0 + \theta_1 x$$

$x$  the input,  $f$  the prediction,  $\theta_0$  the intercept,  $\theta_1$  the slope

$$\mathcal{L}(\theta) = \sum_{k=1}^N (\theta_0 + \theta_1 x_k - y_k)^2$$

$\mathcal{L}$  the loss,  $\theta_0 + \theta_1 x_k$  from the model,  $y_k$  from the label,  $N$  data points

## THE FIT AND ITS RESIDUALS



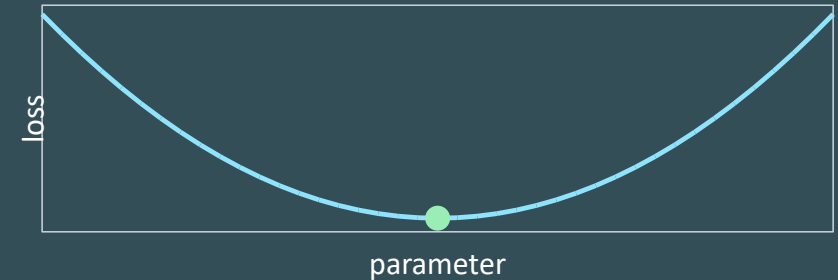
Orange dashes are the residuals. Least squares makes the sum of their squares as small as two parameters allow, which for this data is not small.

# Loss Surface

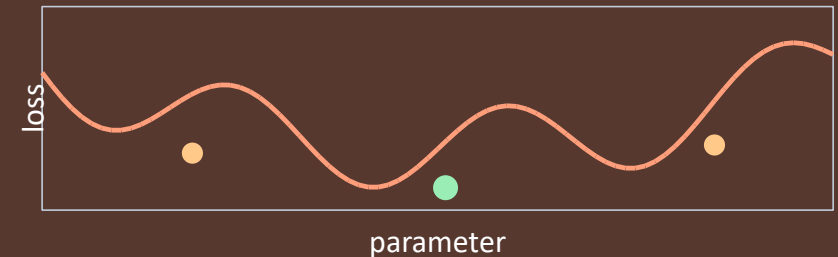
## THE LOSS AS A FUNCTION OF THE PARAMETERS

- **sweep the parameters** — the loss traces a surface over them
- **least squares** — a bowl: one minimum, with a closed-form solution
- **a neural network** — many minima, and no closed form
- **so the minimum has to be searched for** — that search is the optimiser, opened in L6.1
- **the picture to keep** — the bowl is the exception, the rugged surface the rule

### LINEAR MODEL — ONE MINIMUM



### NEURAL NETWORK — MANY MINIMA



# Perceptron and Neuron

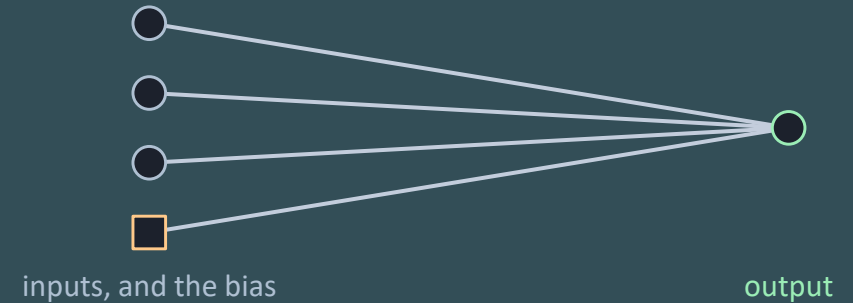
## THE 1958 UNIT AND THE MODERN ONE

|            | PERCEPTRON (1958)            | NEURON (today)                     |
|------------|------------------------------|------------------------------------|
| input      | $x$ , weights $w$ , bias $b$ | the same                           |
| sum        | $z = w \cdot x + b$          | $z = w \cdot x + b$                |
| activation | step: 1 if $z > 0$ , else 0  | continuous: ReLU, tanh, sigmoid    |
| output     | a class, 0 or 1              | a real number                      |
| learning   | error-correction rule        | gradient descent on a loss         |
| represents | one linear boundary          | one linear boundary; layers stack  |
| XOR        | cannot — Ex_04 notebook 01   | one hidden layer can — notebook 01 |

$$y = H(\mathbf{w} \cdot \mathbf{x} + b)$$

$\mathbf{x}$  the inputs,  $\mathbf{w}$  the weights,  $b$  the bias,  $H$  the step function,  $y \in \{0, 1\}$

### ONE NEURON



### CONVERGENCE THEOREM

**if** the classes are linearly separable  
**then** the rule converges, in finite steps  
**if not** it cycles — the case of XOR, next

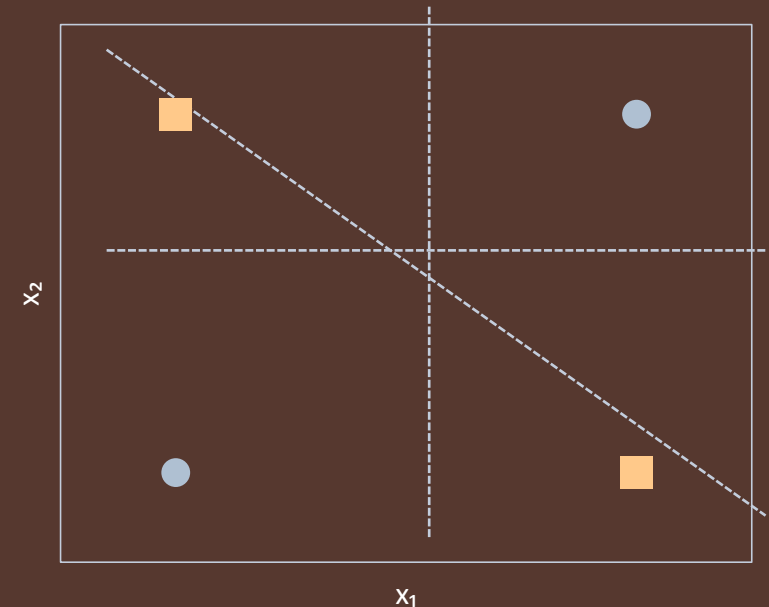
# Linear Separability

## ONE NEURON, ONE STRAIGHT BOUNDARY

- **the weighted sum is zero along a line** — positive on one side, negative on the other
- **AND and OR** — the classes fall either side of some line, so one neuron suffices
- **XOR** — the classes sit on opposite diagonals; no line separates them
- **not a flaw of the learning rule** — no weights represent this function at all

**TALK · 3 MIN** In the XOR plot on the right, try to draw one straight line with the amber squares on one side and the grey circles on the other. Why does every attempt fail?

### XOR — NOT SEPARABLE



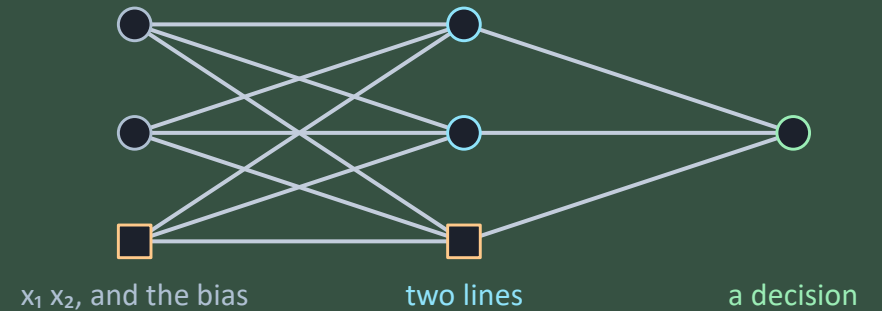
*Amber squares are class 1, grey circles class 0. Every straight line misclassifies at least one point.*

# Hidden Layer

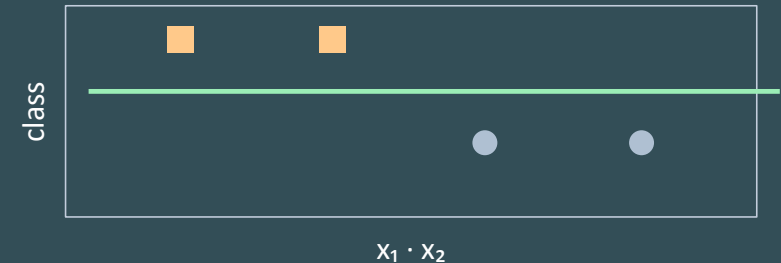
## TWO LINES, THEN A DECISION ABOUT THEM

- **add a hidden layer** — two neurons, each drawing its own line
- **the output neuron** — draws a line in the space of the two hidden decisions
- **there the problem is separable** — the hidden layer has transformed the plane
- **the same in features** — with  $x_1x_2$  as a third input, one line separates XOR
- **Minsky and Papert (1969)** — showed the limit; training a hidden layer came later

### TWO INPUTS, TWO HIDDEN, ONE OUTPUT



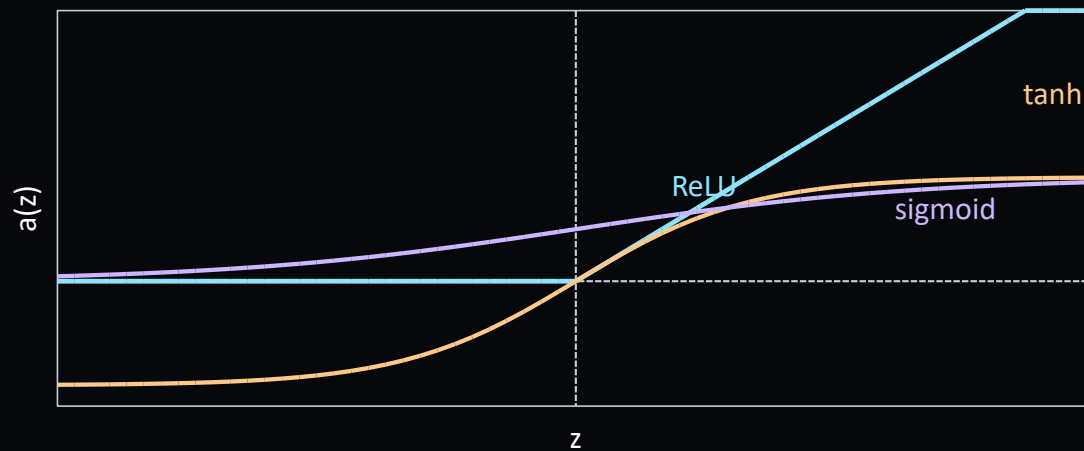
### XOR WITH THE FEATURE $x_1x_2$



# Activation Function

## WHY A NONLINEARITY, AND WHICH ONE

- **without an activation** — a linear function of a linear function is linear, at any depth
- **so the activation is what depth buys** — every hidden neuron has one
- **the choice** — is decided by what the derivatives must carry, in the table on the right



## FUNCTIONS AND DERIVATIVES

|         | $a(z)$             | $a'(z)$               | $a''(z)$          |
|---------|--------------------|-----------------------|-------------------|
| ReLU    | $\max(0, z)$       | 0 or 1                | 0                 |
| tanh    | $\tanh z$          | $1 - \tanh^2 z$       | $\neq 0$ , smooth |
| sigmoid | $1 / (1 + e^{-z})$ | $\sigma (1 - \sigma)$ | $\neq 0$ , smooth |

## WHERE EACH IS USED

**ReLU** classification and regression

**tanh** PDE residuals: second derivatives

**sigmoid** a probability at the output

*A PDE residual holds second derivatives: zero along a ReLU piece, nonzero for tanh.*

# Neural Networks

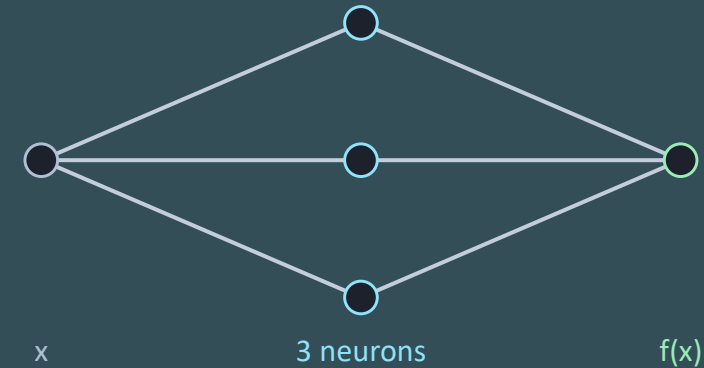
## ONE HIDDEN LAYER, D NEURONS

- **each hidden neuron** — scales the input, shifts it, applies the activation
- **the output** — a weighted sum of the D responses, plus a bias
- **parameters** —  $D_i + 1$  per hidden neuron,  $D + 1$  per output
- **one input, one output** —  $P = 3D + 1$ ; ten neurons is thirty-one parameters

$$f(x, \theta) = \varphi_0 + \sum_{d=1}^D \varphi_d a(\theta_{d0} + \theta_{d1}x)$$

$D$  hidden neurons,  $\theta_{d0}, \theta_{d1}$  the bias and weight of neuron  $d$ ,  $\phi_0, \phi_d$  the output bias and weights

### ONE INPUT, THREE HIDDEN, ONE OUTPUT



### PARAMETER COUNT

$$P = D(D_i + 1) + D_o(D + 1)$$

**D = 10**

31 parameters, one in, one out

# Linear Regions

## D NEURONS, D + 1 LINEAR REGIONS

- **each ReLU neuron** — flat, then rising: one breakpoint
- **D neurons** — D breakpoints, D + 1 linear regions; exactly what the network computes
- **training** — moves the breakpoints; more neurons, more pieces
- **still piecewise linear at D = 10** — smooth from a distance, straight lines up close

**PREDICT · 1 MIN** Compare the two fits on the right. With  $D = 10$ , how many linear regions and how many parameters does the network have, and where does it still miss the dashed target?

*Ex\_04 notebook 03 sweeps D and records the error at each width.*

### D = 3 — FOUR REGIONS



### D = 10 — ELEVEN REGIONS



# Universal Approximation

## THE THEOREM

For any continuous function on a closed bounded region and any tolerance, there is a network with one hidden layer and enough neurons whose output is within that tolerance everywhere on the region.

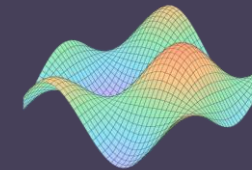
## PIECEWISE APPROXIMATION

- **enough linear regions** — approximate any continuous function to any tolerance
- **the engineering analogy is quantisation** — finer steps, smaller error, more of them
- **what it does not give** — the number of neurons, or a way to find the parameters

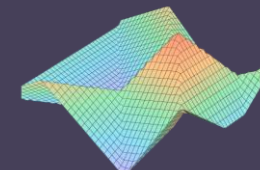
## CHALLENGES

No theory gives the width or the depth. Ex\_04 notebook 04 sweeps both at equal parameter count.

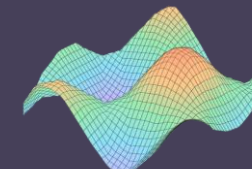
## ONE LAYER OR TWO



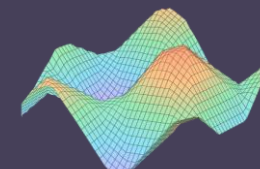
the target function  
 $f(x, y)$  on the unit square



8 neurons, 1 layer  
33 parameters · RMSE 0.102



32 neurons, 1 layer  
129 parameters · RMSE 0.017



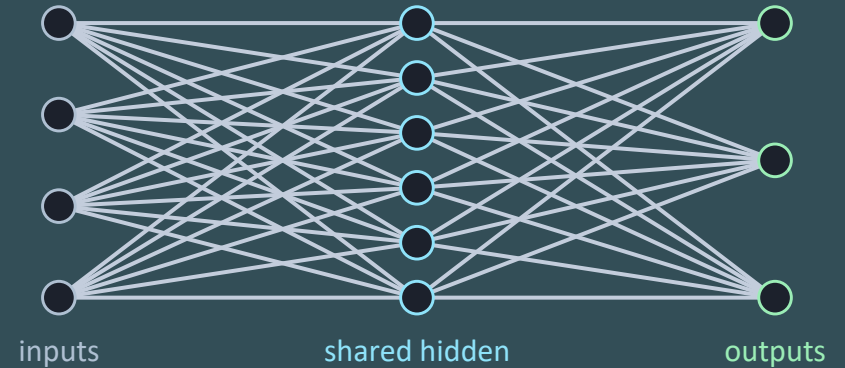
8 + 8 neurons, 2 layers  
105 parameters · RMSE 0.023

# Inputs and Outputs

## SEVERAL OF EACH

- **several inputs** — each neuron cuts the input space with a hyperplane, not a point
- **several outputs** — several output neurons read one shared hidden layer
- **the inputs are what the quantity depends on** — position and time for a temperature field
- **the outputs are what is needed together** — two velocity components and a pressure for a flow
- **the count** —  $P = D(D_i + 1) + D_o(D + 1)$  grows with all three

### FOUR IN, THREE OUT



### SHARED HIDDEN LAYER

Outputs that describe one physical state come from one network. The hidden layer is shared, so the outputs are coupled the way the physics couples them.

# Sizing a Neural Network

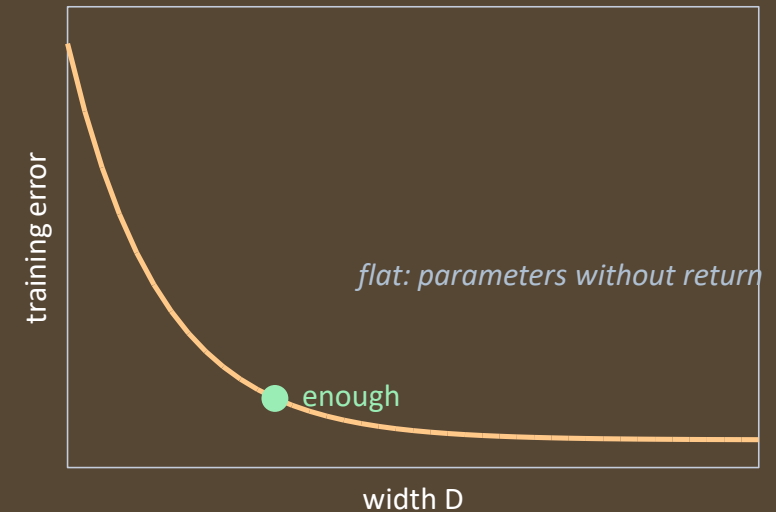
## WIDTH D AND DEPTH N

- **two numbers** — D neurons per layer, N hidden layers; parameters grow as  $N \cdot D^2$
- **start small** — a smooth field on one domain needs tens of neurons, not thousands
- **grow** — only when the training error demonstrably stops falling
- **error against width flattens** — past the elbow, parameters cost time and buy nothing

**TALK · 2 MIN** On the error-against-width curve on the right, where would you stop adding neurons, and if the error were still too high there, what would you try first: more width or more depth?

*Ex\_04 notebook 04 compares deeper and wider at equal parameter count.*

## ERROR AGAINST WIDTH, SCHEMATIC



*Schematic, no units. The curve flattens; the elbow is where a first network should sit, and the notebook finds it for a concrete problem.*

# Challenges

## THE OUTPUT IS A CONSTANT

learning rate · input scaling · dead ReLU units · He initialisation (He 2015)

## THE LOSS IS NaN OR GROWS

normalise inputs and targets · smaller learning rate · gradient clipping (Pascanu 2013)

## TRAINING ERROR SMALL, TEST ERROR LARGE

held-out data · early stopping (Prechelt 1998) · weight decay · a smaller network

## DATA AND ENERGY EFFICIENCY

|                    |                                          |
|--------------------|------------------------------------------|
| <b>print</b>       | input and target ranges, before training |
| <b>watch</b>       | the loss fall in the first epochs        |
| <b>overfit</b>     | five samples on purpose, as a code test  |
| <b>start small</b> | the smallest network that fits           |
| <b>stop</b>        | when held-out error stops falling        |

*Glorot & Bengio (2010) and He et al. (2015) on initialisation; Kingma & Ba (2015) on Adam; Prechelt (1998) on early stopping; UDL ch. 6 to 8.*

# Exercise 4

## FOUR NOTEBOOKS, IN ORDER

- **00** — environment check, read only
- **01** — a perceptron in NumPy: AND, then XOR, then a hidden layer
- **02** — the same network in PyTorch
- **03** — the linear-region experiment: sweep D, record the error



## THE QUESTION EACH NOTEBOOK ANSWERS

- 01** why does one neuron never learn XOR, and what do two hidden neurons change?
- 02** which lines did autograd remove from the NumPy version?
- 03** how does the error fall as D grows, and how many regions do you count?

## REPORT QUESTION

For a problem of your own: how many inputs, outputs and neurons would you start with, and what result would make you add more?

# Key Takeaways

- 1 A neuron is a weighted sum, a bias and an activation**  
Linear regression is the case with one neuron and no activation. Notebook 01, first cells.
- 2 One neuron cannot represent XOR; two hidden neurons can**  
Notebook 01: the single layer never converges, the 2-2-1 network does, with nine parameters.
- 3 A ReLU network with  $D$  neurons is piecewise linear,  $D + 1$  regions**  
Notebook 03: the error falls as  $D$  grows and the fit stays made of straight pieces.
- 4 One hidden layer can approximate any continuous function**  
Given enough neurons. The theorem gives existence; width, depth and training are chosen by you. Notebook 04.
- 5 The activation is chosen by the derivatives the loss needs**  
ReLU for classification and regression; tanh where second derivatives enter the loss.

*Next — L4.2: deep networks, capacity and generalisation.*

# Questions

## TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What is a neuron? Write its equation and name each term in it. *UDL ch. 3.1*
- 2 What separates a perceptron from a neuron, and why does that difference matter for training? *GBC ch. 1.2 and 6.1*
- 3 Why can a single neuron not represent XOR, and what does one hidden layer of two neurons change? *Ex\_04 notebook 01; GBC ch. 6.1*
- 4 What does a ReLU network with  $D$  hidden neurons compute, exactly, and how many linear regions does it have? *UDL ch. 3.2; Ex\_04 notebook 03*
- 5 Why does a network without activation functions reduce to a linear model, whatever its depth? *UDL ch. 3.1*
- 6 Which activation would you choose for a regression problem, and which when second derivatives enter the loss? Why? *UDL ch. 3.1*
- 7 What does the universal approximation theorem state, and what are two things it does not tell you? *UDL ch. 3.2; Cybenko (1989)*
- 8 How many parameters has a network with  $D_i$  inputs, one hidden layer of  $D$  neurons and  $D_o$  outputs? Give the formula. *UDL ch. 3.3 and 3.4*
- 9 Why do several outputs of one physical state share one hidden layer? *UDL ch. 3.4*
- 10 How would you choose the width and depth of a first network for a new problem, and what result would make you change them? *UDL ch. 4.5; Ex\_04 notebook 04*

NEXT

# L4.2 — Deep Neural Networks

---

## BEFORE THE NEXT SESSION

|                    |                                  |
|--------------------|----------------------------------|
| <b>read</b>        | UDL ch. 4 — about forty minutes  |
| <b>run</b>         | Ex_04 notebooks 00 to 03         |
| <b>bring</b>       | the epoch count where XOR failed |
| <b>think about</b> | why nine parameters were enough  |

*Bring the number. L4.2 opens with a show of hands on how long people let the single-layer version run before concluding it would never converge.*